

SOFTWARE ENGINEERING

UNIT – 2

TOPIC – 9

GIT COMMANDS: WORKING WITH REMOTE REPOSITORIES - REMOTE, CLONE, PULL, PUSH, FORK

1. Git Remote

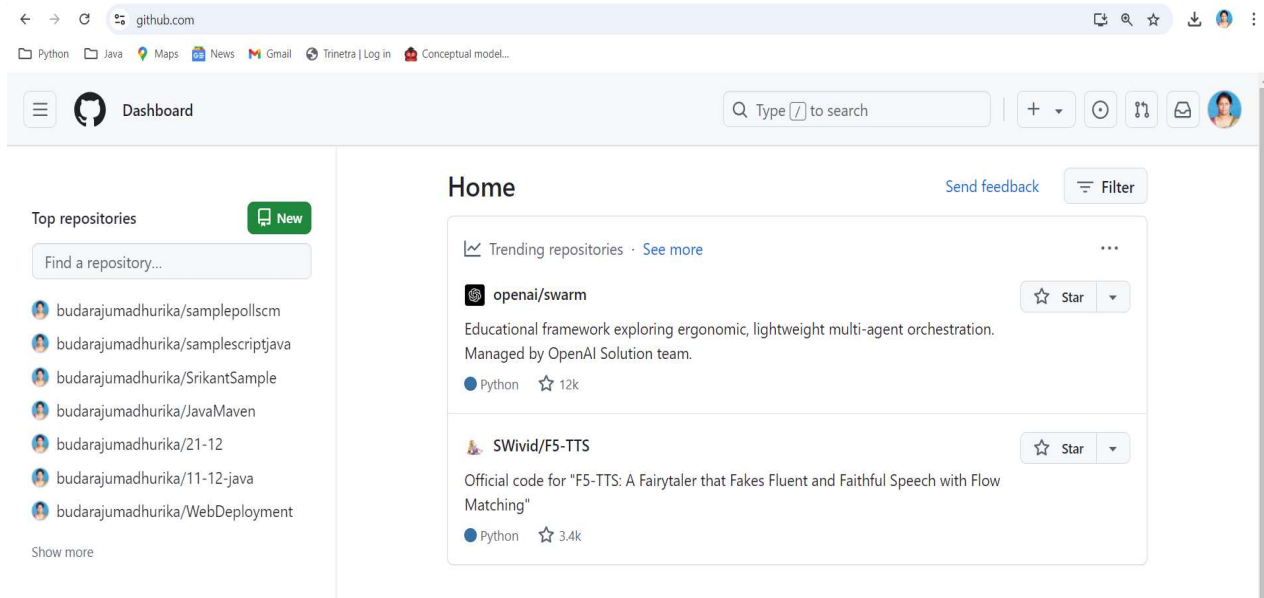
Git remote allows you to manage connections to **remote repositories**, which are Git repositories hosted on the internet (like **GitHub** or **Bitbucket**). A remote repository is where you can save or retrieve your project from a server, and you use this command to add, remove, or view the addresses (URLs) of these repositories.

- To add a new remote repository, you use:

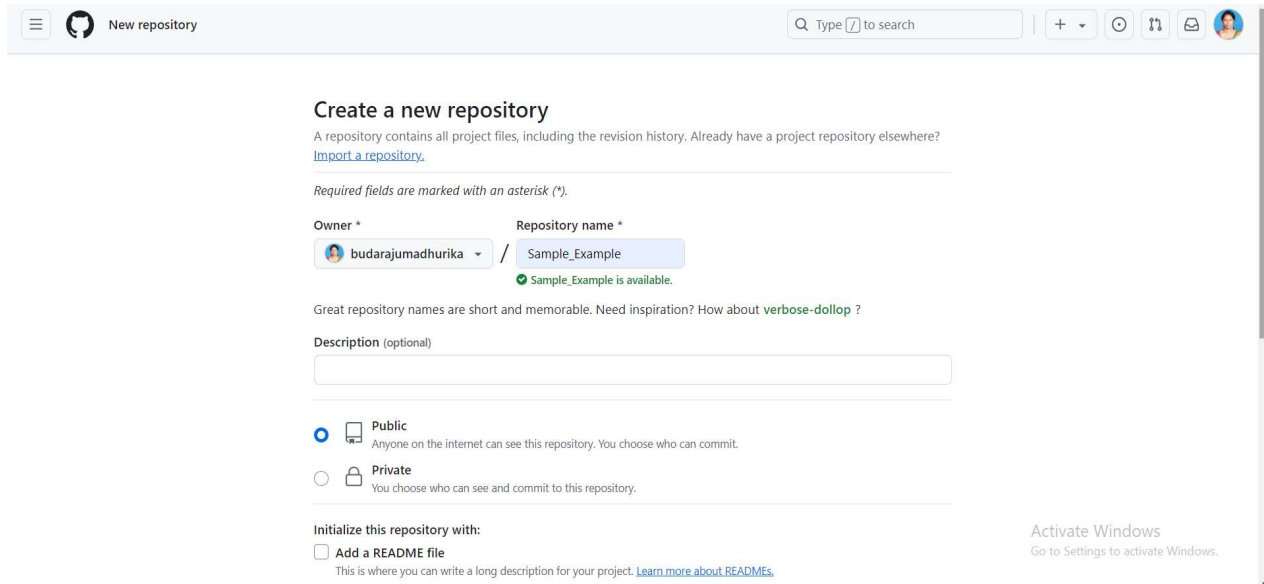
```
git remote add origin https://github.com/user/repo.git
```

This adds a new connection to a remote repository called `origin`, which is the common default name for remote repositories. After setting up the remote, you can start interacting with it, like pushing or pulling code.

- **Important Commands:**
 - `git remote -v`: Shows the current remotes and their URLs.
 - `git remote add <name> <url>`: Adds a new remote connection.
 - `git remote remove <name>`: Removes an existing remote.



The screenshot shows the GitHub Home dashboard. At the top, there's a navigation bar with the GitHub logo, a search bar, and a user profile icon. Below the navigation bar, the main content area is divided into two columns. The left column, titled 'Top repositories', lists several repositories by the user 'budarajumadhurika', including 'samplepollscm', 'samplescriptjava', 'SrikantSample', 'JavaMaven', '21-12', '11-12-java', and 'WebDeployment'. A 'New' button is visible next to the list. The right column, titled 'Home', features a 'Trending repositories' section. It lists 'openai/swarm' with a description 'Educational framework exploring ergonomic, lightweight multi-agent orchestration. Managed by OpenAI Solution team.' and 'SWivid/F5-TTS' with a description 'Official code for "F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching"'. Both repositories show their programming language (Python) and star count (12k and 3.4k respectively). A 'Send feedback' link and a 'Filter' button are also present.



The screenshot shows the 'Create a new repository' page on GitHub. The page has a heading 'Create a new repository' and a subheading 'A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)'. Below this, there's a section for 'Required fields are marked with an asterisk (*)'. The 'Owner' field is set to 'budarajumadhurika' and the 'Repository name' field is set to 'Sample_Example'. A green checkmark indicates 'Sample_Example is available.'. A note says 'Great repository names are short and memorable. Need inspiration? How about [verbose-dollop](#) ?'. The 'Description (optional)' field is empty. Under the 'Initialize this repository with:' section, the 'Public' option is selected, with a note 'Anyone on the internet can see this repository. You choose who can commit.'. The 'Private' option is also available, with a note 'You choose who can see and commit to this repository.'. At the bottom, there's a checkbox for 'Add a README file' with a note 'This is where you can write a long description for your project. [Learn more about READMEs.](#)'. A watermark for 'Activate Windows' is visible in the bottom right corner.

PythonJavaMapsNewsGmailTrinetra | Log inConceptual model...

Public

Anyone on the internet can see this repository. You choose who can commit.

Private

You choose who can see and commit to this repository.

Initialize this repository with:

☐

Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

You are creating a public repository in your personal account.

Create repository

budarajumadhurika / Sample_Example

Type to search

<>CodeIssuesPull requestsActionsProjectsWikiSecurityInsightsSettings

Sample_Example

Public

Pin

Unwatch 1

Fork 0

Star 0

Set up GitHub Copilot

Use GitHub's AI pair programmer to autocomplete suggestions as you code.

Get started with GitHub Copilot

Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop

or

HTTPS

SSH

https://github.com/budarajumadhurika/Sample_Example.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Sample_Example" >> README.md
git init
git add README.md
```

Activate Windows

Go to Settings to activate Windows.

3

The screenshot shows a Windows desktop environment. At the top, there's a taskbar with icons for Python, Java, Maps, News, Gmail, and Trinetra | Log in. Below the taskbar, there are two panels: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. The main area displays a 'Quick setup' guide for creating a new repository on the command line. The guide includes a code block with the following commands:

```
echo "# Sample_Example" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/budarajumadhurika/Sample_Example.git
git push -u origin main
```

Below the code block, there's a section for '...or push an existing repository from the command line'. At the bottom, a file explorer window is open, showing the path: This PC > Desktop > AY-23-24-Sem-2 > RKR21-SE > Tesseract > Sample_Example. The file explorer shows a single file named 'a.txt' with a size of 1 KB. A context menu is open over the file, showing options like 'View', 'Sort by', 'Group by', 'Refresh', 'Customize this folder...', 'Paste', 'Paste shortcut', 'Undo Rename', 'Git GUI Here', 'Git Bash Here', 'Open with Code', 'Give access to', 'New', and 'Properties'.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch -M main

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote add origin https://github.com/budarajumadhurika/Sample_Example.git

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote -v
origin https://github.com/budarajumadhurika/Sample_Example.git (fetch)
origin https://github.com/budarajumadhurika/Sample_Example.git (push)
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
seract/Sample_Example (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 248 bytes | 82.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/budarajumadhurika/Sample_Example.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

github.com/budarajumadhurika/Sample_Example

Python News Gmail Trinetra | Log in Conceptual model...

budarajumadhurika / Sample_Example

Type [Z] to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Sample_Example Public

Pin Unwatch 1 Fork 0 Star 0

Set up GitHub Copilot

Use GitHub's AI pair programmer to autocomplete suggestions as you code.

Get started with GitHub Copilot

Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH https://github.com/budarajumadhurika/Sample_Example.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Sample_Example" >> README.md
git init
git add README.md
```

Activate Windows
Go to Settings to activate Windows.

budarajumadhurika / Sample_Example

Type [Z] to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Sample_Example Public

Pin Unwatch 1 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

budarajumadhurika Added new file into master eaafdb9 - yesterday 1 Commit

a.txt Added new file into master yesterday

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

1 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

Activate Windows

[Publish your first package](#) to Settings to activate Windows.

2. Connecting Git to GitHub Using SSH Key

SSH (Secure Shell) keys are a way to connect your computer to GitHub without needing to enter a password each time.

It uses a **key pair**:

- A **private key** that stays on your computer.
- A **public key** that is shared with GitHub to verify your identity.
- **Steps for Generating SSH Key:**
 1. **Generate SSH Key:** Use the command `ssh-keygen -t rsa` to generate the key on your system.
 2. **Copy the Public Key:** The public key is stored in a file (usually `id_rsa.pub`), and you copy this key.
 3. **Add to GitHub:** In GitHub, go to **Settings > SSH and GPG keys > New SSH Key**. Paste the copied key into the text box and click “**Add SSH key**.”

This allows you to securely interact with GitHub without typing your password every time.

```

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
seract/Example (master)
$ ssh-keygen -t rsa -C budarajumadhurika@gmail.com
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/Madhu/.ssh/id_rsa):
/c/Users/Madhu/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/Madhu/.ssh/id_rsa
Your public key has been saved in /c/Users/Madhu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:97Oywwl05Gbky soc5xws7eHzse5JjD1CRPH4k7idFr4 budarajumadhurik
a@gmail.com
The key's randomart image is:
+---[RSA 3072]-----+
|
|      =.
|     B o
|    . @ .
|   o O + .
|  S B * =
| o * B X +
| + . % @
| .. O *
| o+.oE
+-----[SHA256]-----+

```

This PC > Local Disk (C:) > Users > Madhu > .ssh

id_rsa
Type: File
Date modified: 10/17/2024 1:27 PM
Size: 2.56 KB

id_rsa.pub
Type: Microsoft Publisher Document
Date modified: 10/17/2024 1:27 PM
Size: 581 bytes

known_hosts
Type: File
Date modified: 6/30/2024 8:27 PM
Size: 1.36 KB

known_hosts.old
Type: OLD File
Date modified: 9/23/2022 1:25 AM
Size: 432 bytes

Open
Edit
New
Print
Share with Skype
Open with Code
Edit with Notepad++
Scan with Microsoft Defender...
Share
Open with
Give access to
Add to archive...
Add to "id_rsa.rar"
Compress and email...
Compress to "id_rsa.rar" and email
Restore previous versions
Send to
Cut
Copy
Create shortcut
Delete
Rename
Properties

Notepad
Publisher
Search the Microsoft Store
Choose another app

id_rsa.pub - Notepad
File Edit Format View Help

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDS7JLwInc+0M00+SONcQV3ZlhFOWmYrcdQlh6okfehK9H8Z+
```

github.com/settings/ssh/new

Python Java Maps News Gmail Trinetra | Log in Conceptual model...

Settings

Type to search

B MADHURIKA (budarajumadhurika)
Your personal account

Go to your personal profile

Public profile
Account
Appearance
Accessibility
Notifications

Access

Billing and plans
Emails
Password and authentication
Sessions

SSH and GPG keys

Organizations
Enterprises
Moderation

Add new SSH Key

Title

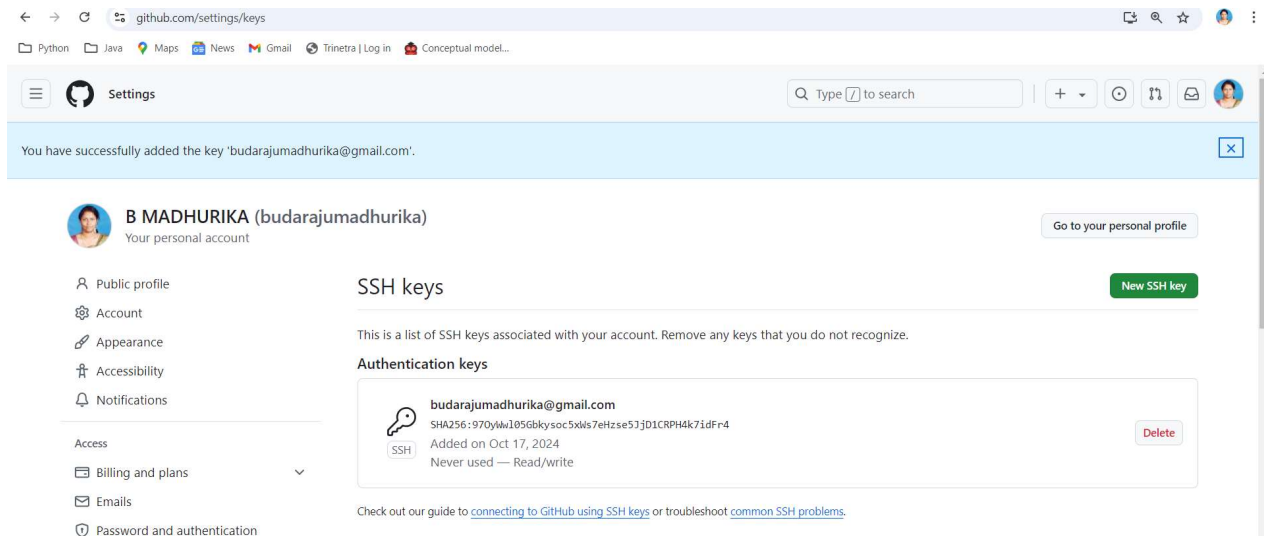
Key type
Authentication Key

Key

```
ssh-rsa  
AAAAB3NzaC1yc2EAAAADAQABAAQgQDS7JLwInc+0M00+SONcQV3ZlhFOWmYrcdQlh6okfehK9H8Z+VoR4fwnDQIEJYnBhxUUn1gXgaktWO8zSnj  
E2KOHrCkxNvrtzHqLRYEHxG+VVvD3iWKT86QJT/btUNk6gguwB4kExsPBENESCtmuImAG8vUHVefS7YGqKc9sMacVxZhp4ix5sioJrITXDB6H5d5iRWRV  
QlpOUpS+ik2KwJnHVeVon3Za+qaxnkQXvsXoTXGfQhe29p9ucHgSWquBly11BN9x4F3WIFma1L54Its9jpQX5NyMGIQyTAKpfXwMH3zH2Hp36Z9Uzd  
5gssGyakg8T2frEcl4dmCPWEh7/ravfHuayN7LODqR0IIPChXgJlyi5EK52VpspQaF0KvkiTS6Uv1107sl2mVhNsaQWp/Wkfu6U/mdHzz7OuGNAM2fWn  
XD4n7bz9EQnkoO9+z84uiBKgnFxsZc1Yf/Tp9qvV9XUEZLXnp3bbj3OWE81xPQH5u+zDWgvTMSiKgCYd7s= budarajumadhurika@gmail.com
```

Activate Windows
Go to Settings to activate Windows.

Add SSH key



3. Git Clone

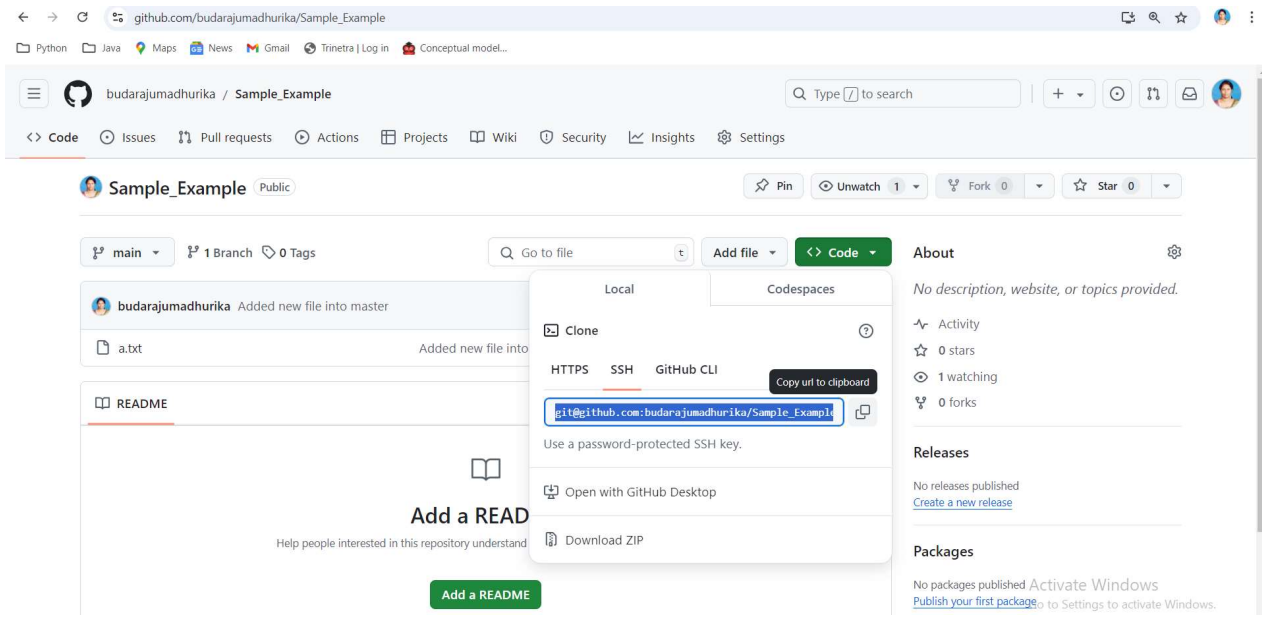
The **git clone** command is used to **download** a copy of a project from a remote repository to your computer. It creates a local version of the project that you can start working on right away.

- If you want to work on a project hosted on GitHub, you would use:

```
git clone https://github.com/user/repo.git
```

This creates a local copy of the project so you can begin editing the code on your computer.

- **Other Commands:**
 - **ls -ltr**: Lists the files in the current directory, showing the latest modified files after the cloning is done.

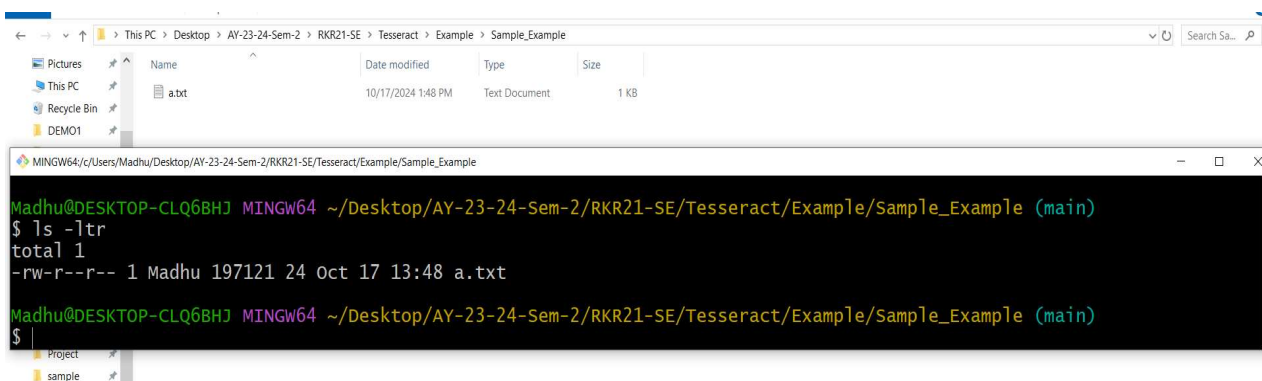


```
MINGW64/c/Users/Madhu/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
seract/Example (master)
$ git clone git@github.com:budarajumadhurika/sample_example.git
Cloning into 'sample_example'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from
0)
Receiving objects: 100% (3/3), done.
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example (master)
$ ls -ltr
total 0
drwxr-xr-x 1 Madhu 197121 0 Oct 17 13:48 sample_example/

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example (master)
$ cd sample_example

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/sample_example (main)
$
```



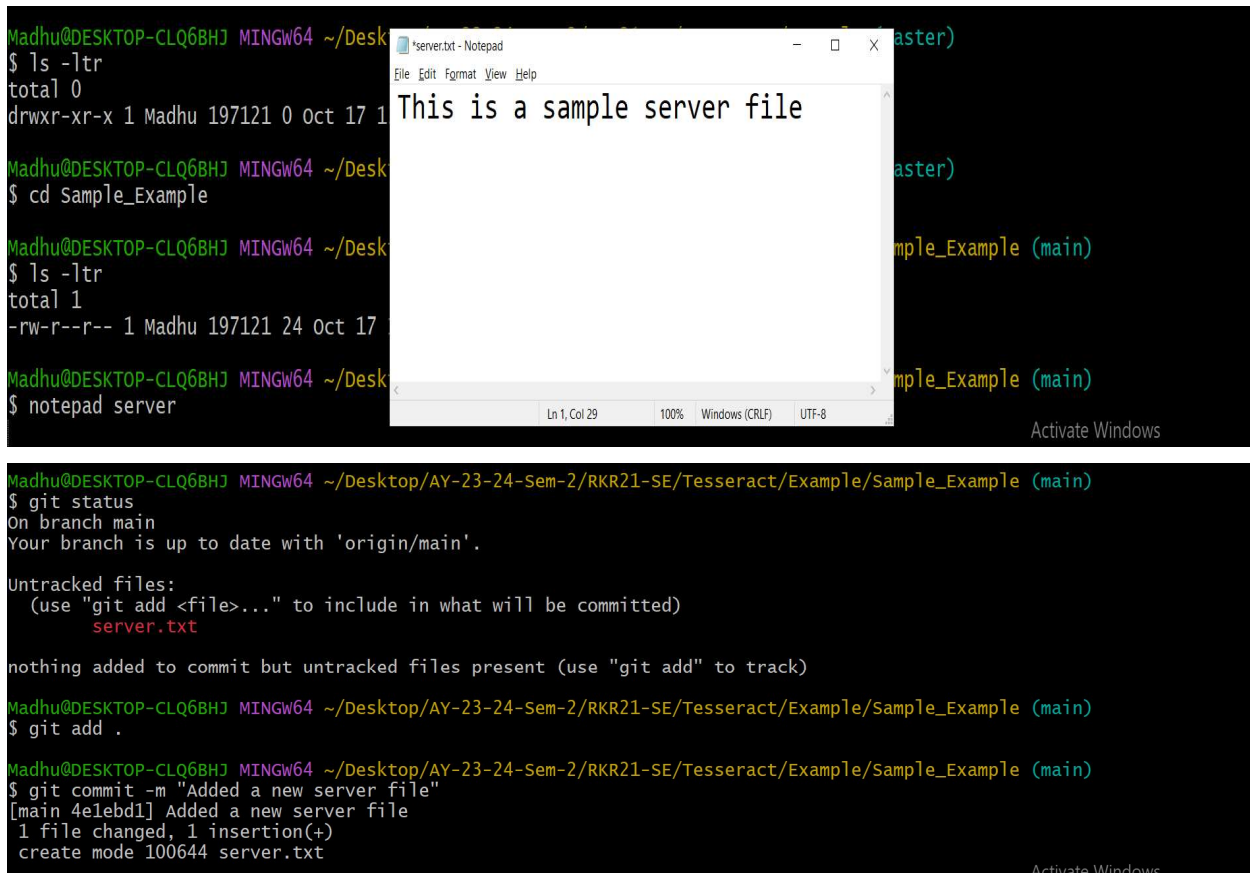
4. Git Push

After making changes locally, you use the **git push** command to send these changes from your local repository to the **remote repository** (for example, back to GitHub). This is how you **share your work** with others or keep your remote repository updated with your changes.

- After editing files on your computer and committing the changes, you would push them to the remote repository using:

```
git push origin master
```

This pushes the changes from your local `master` branch to the `origin` remote (which is the GitHub repository).



The screenshot shows a terminal window with the following commands and output:

```
Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ ls -ltr
total 0
drwxr-xr-x 1 Madhu 197121 0 Oct 17 11:00 .
drwxr-xr-x 1 Madhu 197121 0 Oct 17 11:00 ..

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ cd Sample_Example

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ ls -ltr
total 1
-rw-r--r-- 1 Madhu 197121 24 Oct 17 11:00 server.txt

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ notepad server
```

A Notepad window titled "server.txt - Notepad" is open, showing the text:

```
This is a sample server file
```

Below the terminal window, the output of the following commands is shown:

```
Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       server.txt

nothing added to commit but untracked files present (use "git add" to track)

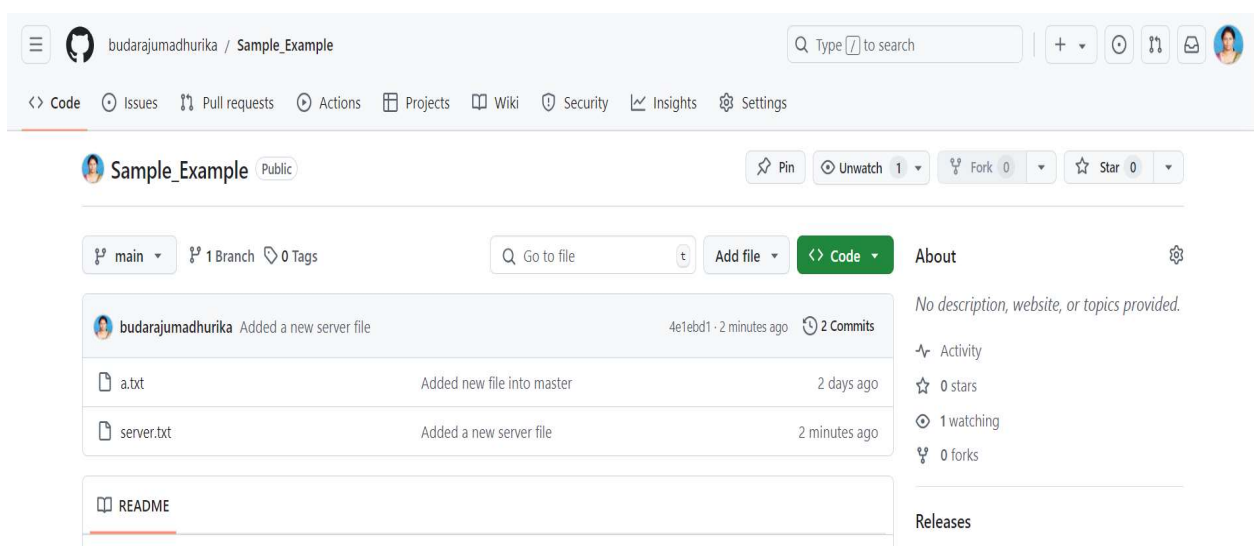
Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git add .

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git commit -m "Added a new server file"
[main 4e1ebd1] Added a new server file
1 file changed, 1 insertion(+)
create mode 100644 server.txt
```

```
Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git remote -v
origin git@github.com:budarajumadhurika/Sample_Example.git (fetch)
origin git@github.com:budarajumadhurika/Sample_Example.git (push)

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:budarajumadhurika/Sample_Example.git
   eaafdb9..4e1ebd1  main -> main

Madhu@DESKTOP-CLQ68HJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ |
```



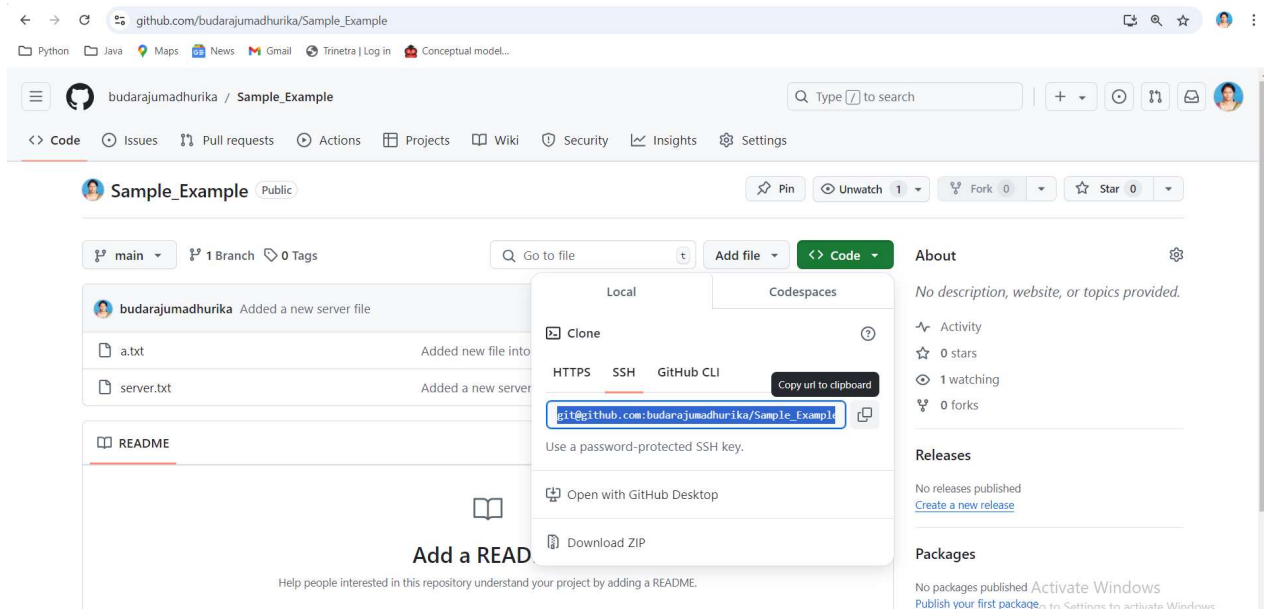
5. Git Pull

The **git pull** command fetches changes from the remote repository and merges them into your local repository. This ensures that you have the latest version of the project, including updates made by your teammates or collaborators.

- If you're working on a project with others and want to get the latest updates they made on GitHub, you would use:

```
git pull origin master
```

This will download any new commits from the `master` branch of the remote repository (`origin`) and merge them with your local code.



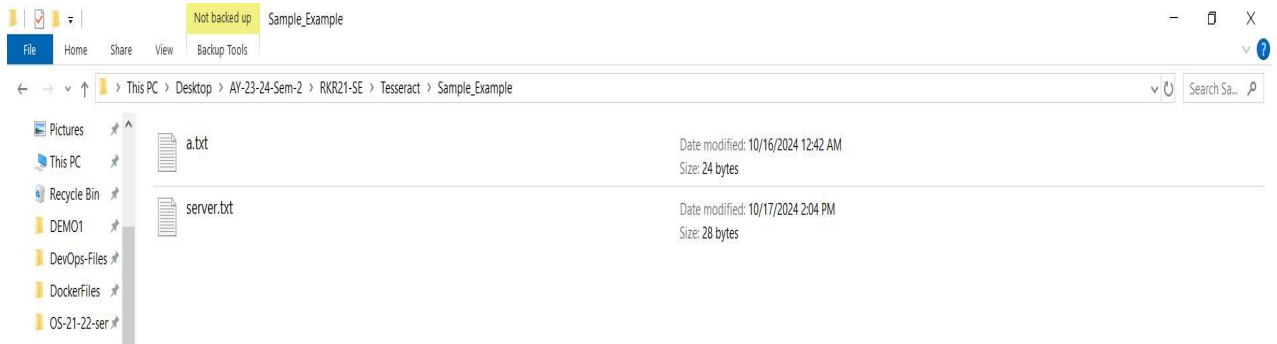
```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote -v
origin https://github.com/budarajumadhurika/Sample_Example.git (fetch)
origin https://github.com/budarajumadhurika/Sample_Example.git (push)

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ ls -ltr
total 1
-rw-r--r-- 1 Madhu 197121 24 Oct 16 00:42 a.txt
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 294 bytes | 21.00 KiB/s, done.
From https://github.com/budarajumadhurika/Sample_Example
 * branch            main       -> FETCH_HEAD
    eaafdb9..4e1ebd1  main       -> origin/main
Updating eaafdb9..4e1ebd1
Fast-forward
 server.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 server.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ ls -ltr
total 2
-rw-r--r-- 1 Madhu 197121 24 Oct 16 00:42 a.txt
-rw-r--r-- 1 Madhu 197121 28 Oct 17 14:04 server.txt
```

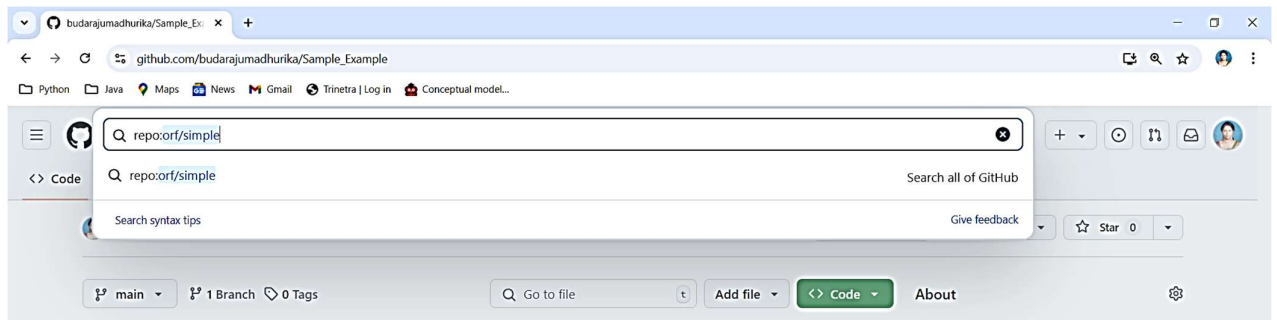
Activate Windows



6. Git Fork

A **fork** is a personal copy of someone else's repository, typically used in open-source projects. When you **fork** a repository, you get your own version of the project on GitHub that you can edit without affecting the original project. You can then propose changes to the original project by making a **pull request**.

- To fork a repository, go to the original repository on GitHub and click the **"Fork"** button. This makes a copy of the repository under your GitHub account where you can make your own changes.



github.com/orf/simple

Python Java Maps News Gmail Trinetra | Log in Conceptual model...

orf / simple

Type [] to search

Code Issues 4 Pull requests 2 Actions Projects Wiki Security Insights

This repository has been archived by the owner on Oct 26, 2018. It is now read-only.

simple Public archive

Watch 27 Fork 178 Star 505

Fork your own copy of orf/simple

master 4 Branches 0 Tags

Go to file

Code

About

Simple is a clone of Obtvs written in Python running on Flask.

Readme MIT license Activity 505 stars 27 watching 178 forks Report repository

orf Merge pull request #54 from adamchainz/patch-1 f64b392 · 6 years ago 278 Commits

simple	Change url protocol for more secure access	8 years ago
.gitignore	Bump version and fix nginx_config from not working	10 years ago
LICENSE	Add MIT license	10 years ago
MANIFEST.in	Add Manifest.in	10 years ago
README.md	Update README for maintenance status	6 years ago

orf / simple

Type [] to search

Code Issues 4 Pull requests 2 Actions Projects Wiki Security Insights

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

No available destinations to fork this repository.

[budarajumadhurika/simple](#)

github.com/budarajumadhurika/simple

Python Java Maps News Gmail Trinetra | Log in Conceptual model...

Code Pull requests Actions Projects Wiki Security Insights Settings

simple Public

forked from orf/simple

Pin Watch 0 Fork 0 Star 0

master

Go to file

Code

About

Simple is a clone of Obtvs written in Python running on Flask.

Readme MIT license Activity 0 stars 0 watching 0 forks

Activate Windows
Go to Settings to activate Windows.

Releases

This branch is up to date with orf/simple:master. Contribute Sync fork

orf Merge pull request orf#54 from adamchai... f64b392 · 6 years ago 278 Commits

simple	Change url protocol for more secu...	8 years ago
.gitignore	Bump version and fix nginx_config...	10 years ago
LICENSE	Add MIT license	10 years ago
MANIFEST.in	Add Manifest.in	10 years ago

Detailed Flow with Examples

1. **Clone a Repository:** If you want to contribute to an open-source project or collaborate with others, you first clone the repository to your computer:

```
git clone https://github.com/user/repo.git
```

2. **Make Changes Locally:** Once you have the project on your local machine, you can make changes to the files, such as fixing bugs or adding new features. After editing, you commit the changes:

```
git commit -m "Added new feature"
```

3. **Push Changes to Remote:** After committing your changes, you push them back to the remote repository (GitHub):

```
git push origin master
```

4. **Pull Changes Made by Others:** If someone else makes changes to the project while you're working on it, you can pull those updates to ensure your version is up-to-date:

```
git pull origin master
```

5. **Use SSH Keys for Security:** Instead of typing your password every time you push or pull changes, you can set up an SSH key to make this process secure and automatic:

```
ssh-keygen -t rsa # Generate SSH key
```

6. **Forking a Repository:** If you want to make your own changes to someone else's project (such as in open-source), you fork the repository first, creating your own copy:
 - Visit the original project on GitHub.
 - Click the **Fork** button.
 - Make your changes in your personal copy, and then propose your updates back to the original repository by creating a **pull request**.

- **git fetch :**

git fetch is a **safe** Git command that retrieves the latest changes from a **remote repository** into your **local repository**—but it **does NOT merge** or change your working files.

You can later merge these changes after reviewing.

When to Use git fetch

- Before starting work to **see if teammates pushed changes**.
- During collaboration to **review changes** before integrating.
- In CI/CD systems or automation pipelines.

SYNTAX : git fetch

Downloads latest changes like – new branches, new commits on existing branches etc. but does not change the working directory.

git branch -r

Show remote branches to check what branches are fetched

git log HEAD..origin/main --oneline

What commits are present in the remote branch but not in your local branch?

Example : git log HEAD..origin/main --oneline

00ec094 (origin/main, origin/HEAD) Update 2 login.html

git diff main origin/main

This shows what's different between your local main and remote main.

View differences between the two files

git diff HEAD origin/main -- login.html

view changes in login.html

or

git show origin/main:login.html

view the remote version of login.html

Once you are satisfied with the changes you can merge the changes

git merge origin/main

Alternate

git pull # Shortcut for fetch + merge

Git Patch: A Powerful Tool for Sharing and Applying Changes

A patch file contains the diffs between two versions of a file or set of files. It's a way to capture changes made in a commit, so you can:

- Send them to others (via email or other means).
- Apply them to a different repository (or different branch).

Key Benefits of using Patches:

- **Offline Contributions:** You can make changes without pushing them to a remote repo.
- **Easy Collaboration:** Share small changes as patches instead of full repositories.
- **Lightweight:** Patches are lightweight and easy to transfer via email.
- **Open Source Contributions:** Submit patches to a project without direct commit access.
- **Code Reviews:** Share changes with team members for review without pushing to a remote.
- **Collaborative Development:** When working in a restricted environment (e.g., no internet), patches can be used to exchange changes.
- **Backup of Changes:** Save patches as backups before applying risky changes.

Creating a Git Patch

git format-patch : creates a patch. Generates one or more .patch files

git format-patch -1 HEAD -1 refers to last commit
 HEAD refers to current commit
(creates a . patch file with the name **0001-<commit-message>.patch**)

git format-patch HEAD~3 Patches for the last 3 commits
(generates 3 .patch files, each corresponding to one commit)

You can send a .patch file to someone via email, file sharing, or any other method.

Applying a Git Patch

git apply : This will apply the changes from the .patch file to your working directory (but not commit them yet).

git apply <patch-file>

Inspect the changes, stage and commit the changes to your repository.

Facilitating Collaborative Work

Git allows you to add collaborators to your repository - **Collaborators** are **people who are given direct access** to your GitHub repository so they can: Read the code, Make changes (write access), Create and push branches, Review and merge pull requests (if permissions allow), Manage issues and pull requests.

Steps to Add Collaborators on GitHub

1. Go to Your Repository

- Log in to GitHub.
- Navigate to your repository:
<https://github.com/<your-username>/<your-repo>>

2. Click on the "Settings" Tab

- You'll see it at the top or right corner of the repo (if you're the owner).

3. Click on "Collaborators" or "Manage Access"

- Under the left sidebar (or in **Settings > Access** for newer GitHub layout).

4. Click "Invite a collaborator"

- Enter their **GitHub username** or **email**.
- Select the user from the dropdown.

5. Choose Their Role

You can assign roles like:

- **Read** – Can view code or **Write** – Can push commits etc.

6. Click Send Invitation

- The invited user will get a notification and must **accept** the invitation.

Viewing/Managing Collaborators

- In the same **Settings > Collaborators** page, you can:
 - Remove collaborators
 - Change their roles

When Collaborators Push contents:

A collaborator makes changes to files, commits them, and then push the contents,Their commits are **added to the remote repo**. If it's a **shared branch like main**, everyone else must pull the latest changes using **git pull**.

Note: Most teams **work on feature branches** and create **Pull Requests (PRs)** instead of pushing directly to main. (as it is risky....merge conflicts.. unwanted/non-reviewed updated).

How to add reviewers to a Pull Request

Pull Requests allow team members to **review** code before merging into the main branch.

Steps to Add Reviewers:

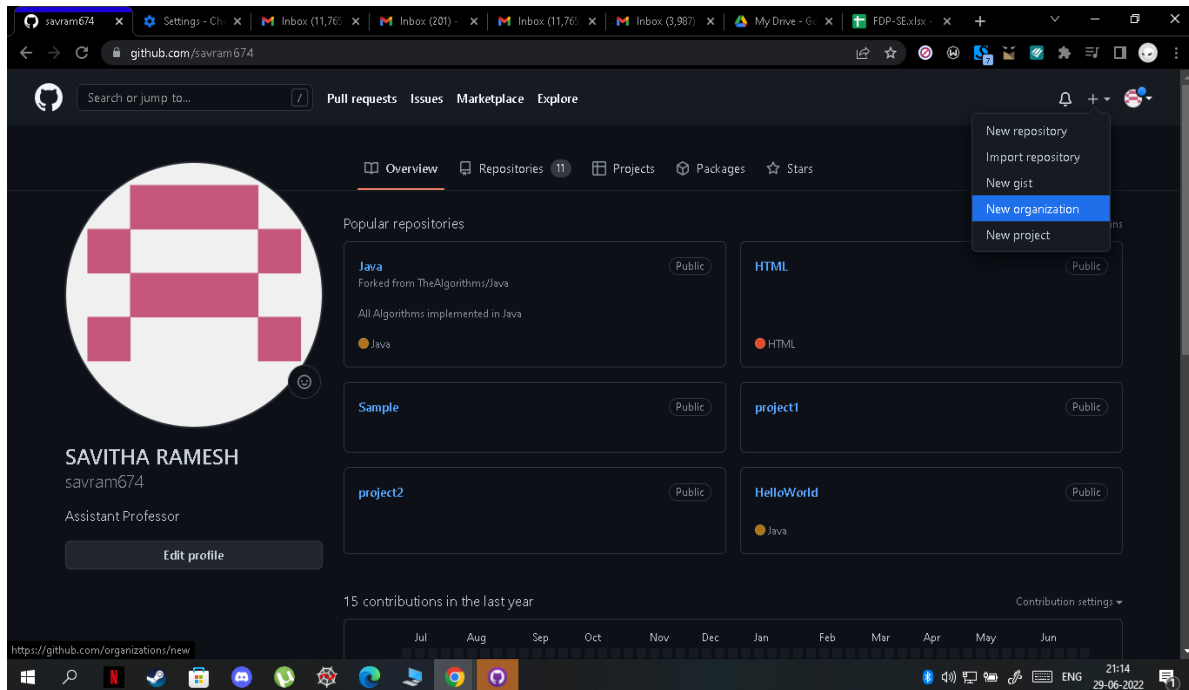
1. Push your changes to a new branch: assume its in feature-branch,
`git push origin feature-branch`
 2. Go to GitHub → Your Repository → Click "**Compare & pull request**"
 3. In the PR form:
 - Add title and description.
 - On the right sidebar, find **Reviewers** section.
 - Search and select collaborators' names.
 4. Click **Create Pull Request**.
- Selected reviewers get notified.
 - They can **approve**, **request changes**, or **comment**.

Note:

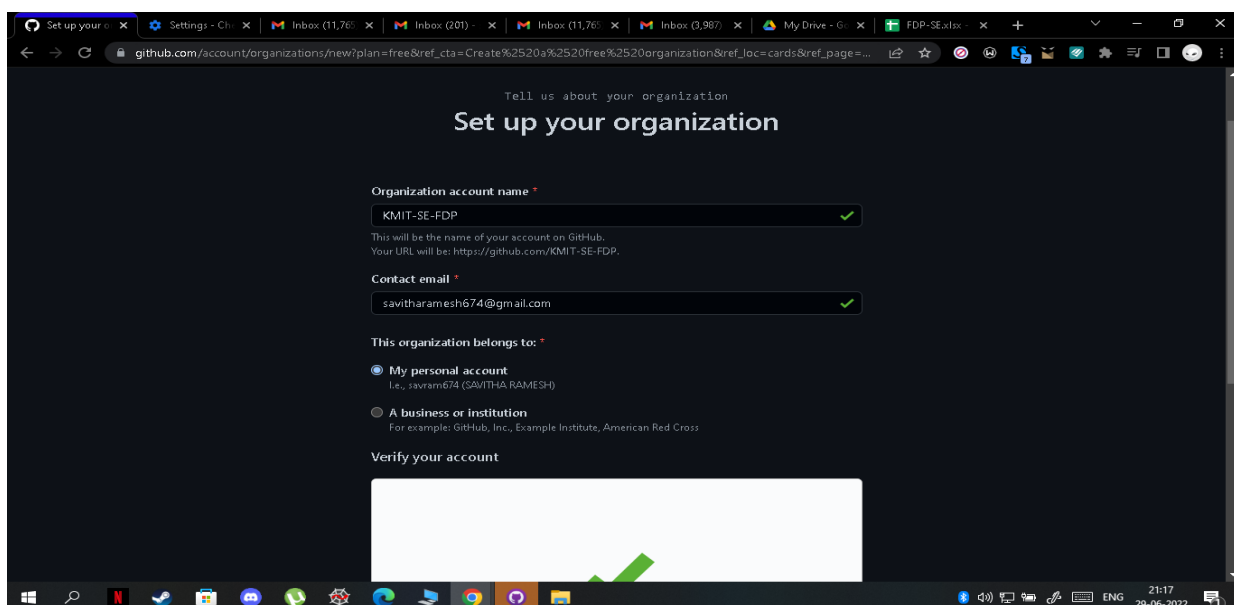
On GitHub, you can assign labels to PRs/issues for easy tracking. Example “bug”, “enhancement”, "needs review" etc.

Collaboration Using Organization:

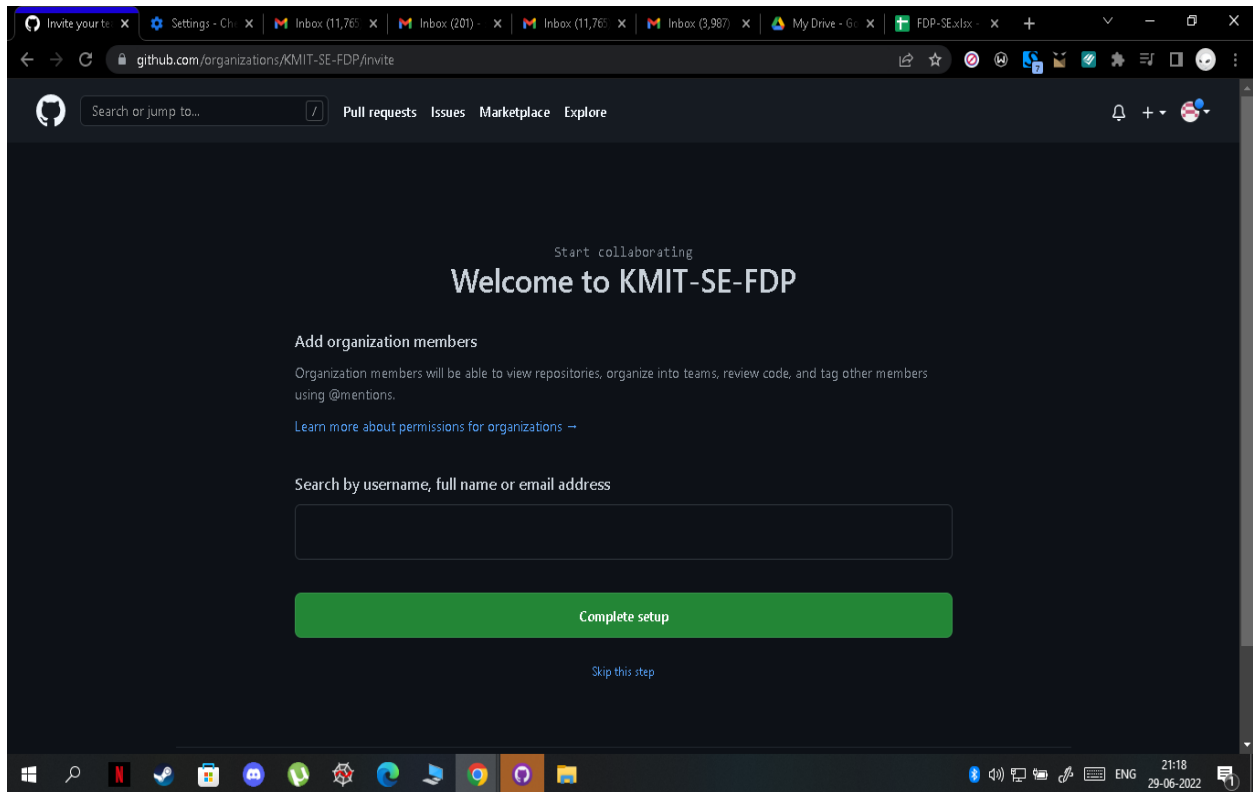
Step 1: Create a new Organization - Organizations are shared accounts where businesses and open-source projects can collaborate across many projects at once. Owners and administrators can manage member access to the organization's data and projects with sophisticated security and administrative features.



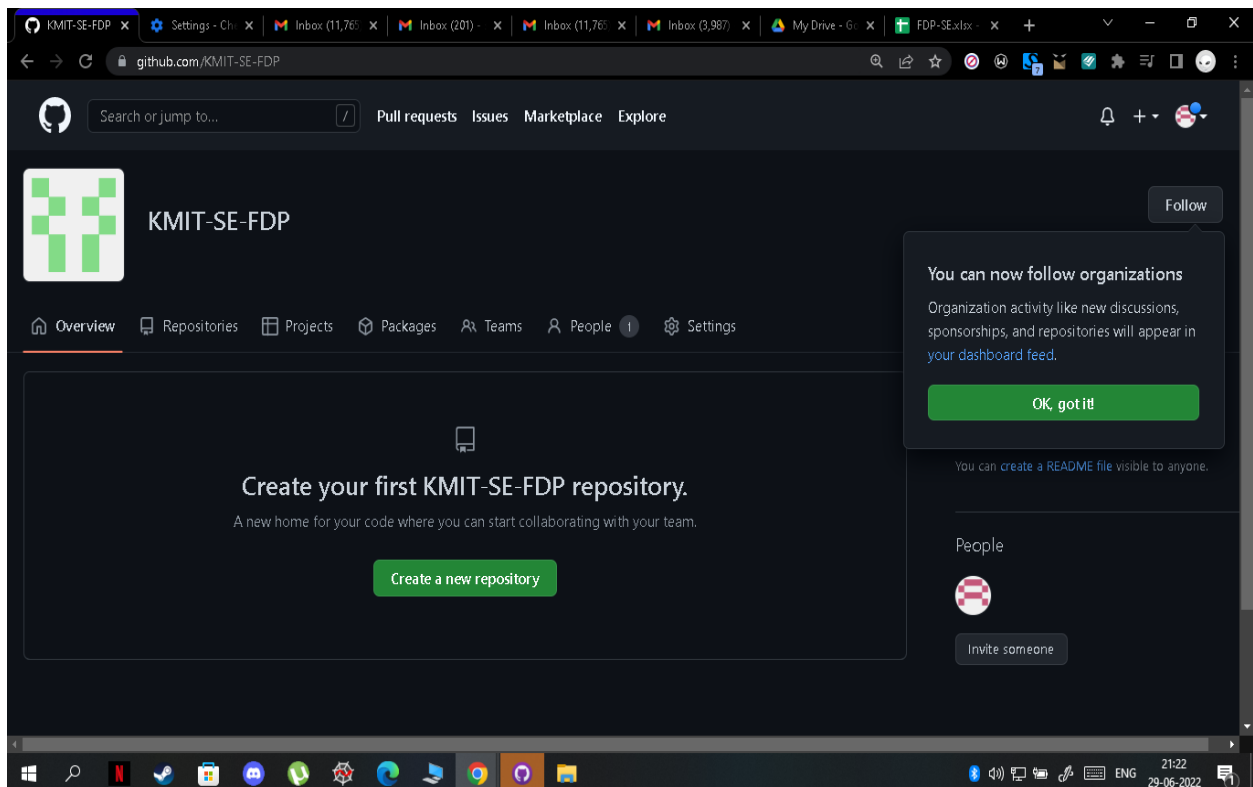
Step 2: Set up your organization by entering the details like name, associated email, account verification, inviting or adding members to the organization, etc.

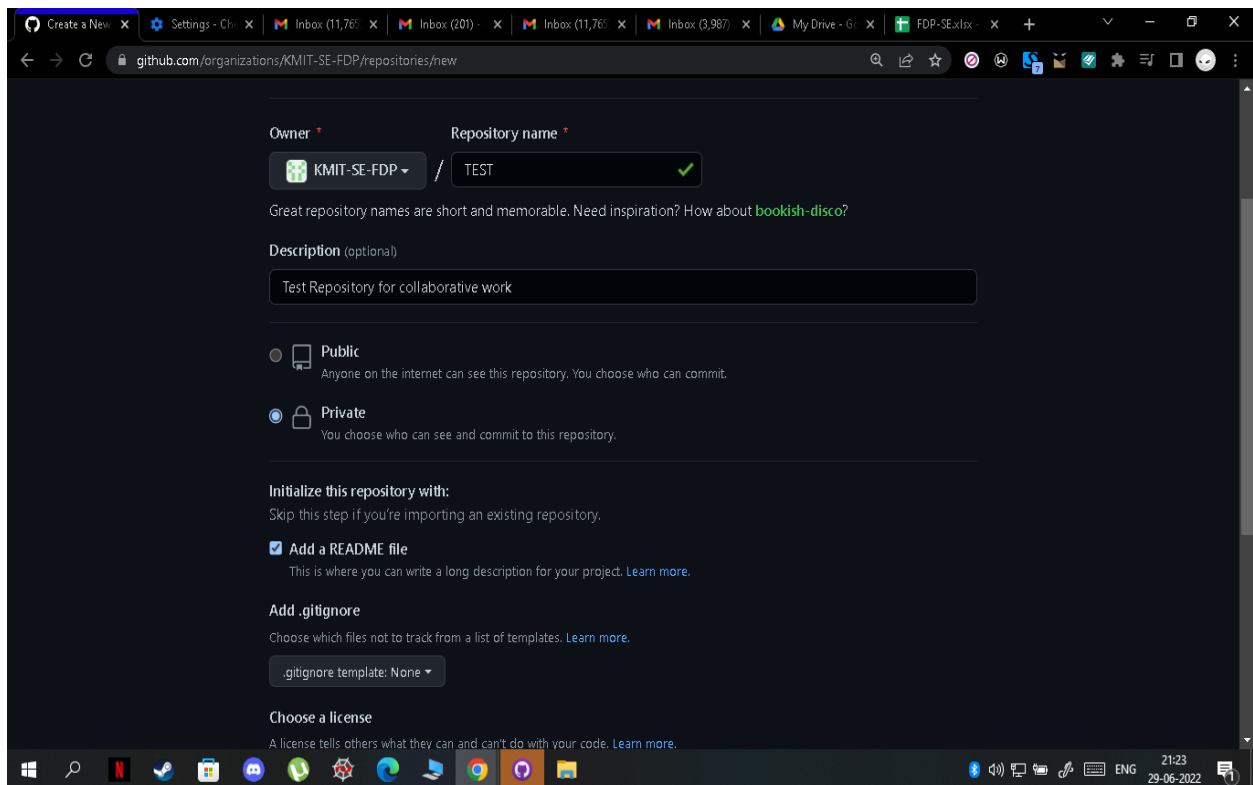


Step 3 : Complete the setup by adding members to the group



Step 4 : Create the remote repository for storing the project related files. This repository is accessible to every member of the team as per the permissions given.





The repository can be made private so that it is accessible only to the group members rather than being in a public domain.

Now all the members of the team can contribute to the development of the project and the different files with all the versions and modification notices will be available in the respective repositories and is accessible to all the members

